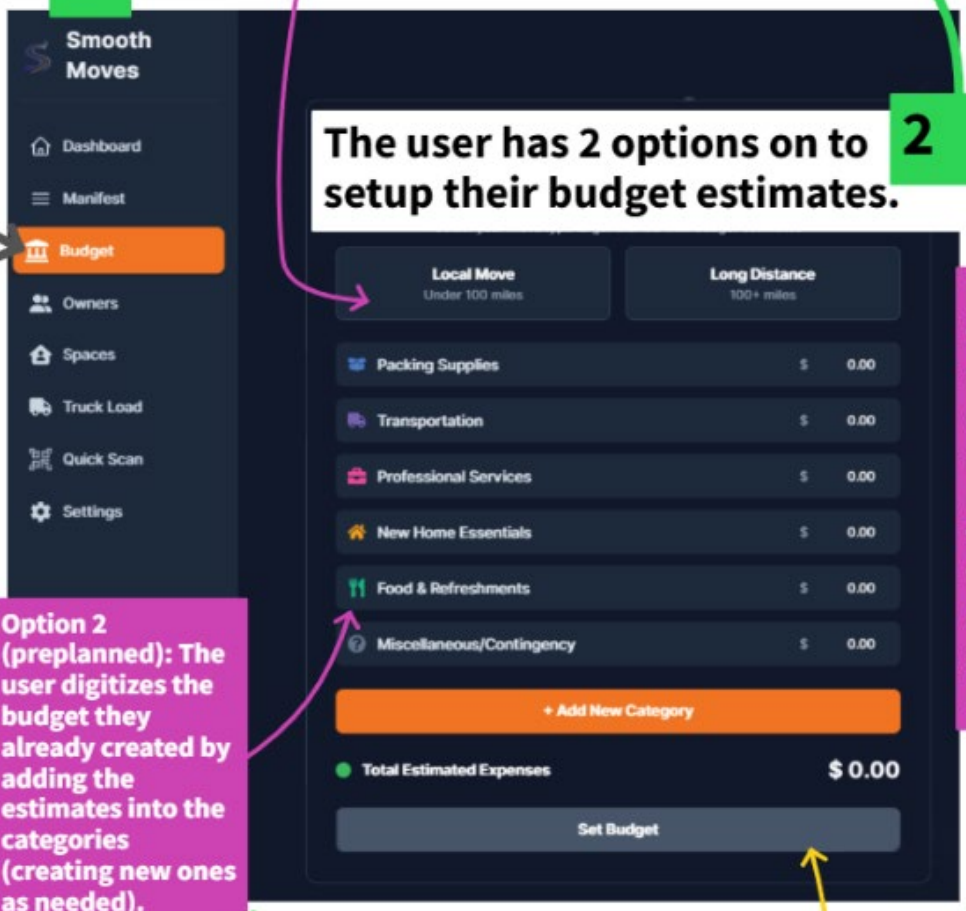
If there has not been a budget set, when the user taps this (for the 1st time usually) they are directed to the setup modal that is displayed over the empty background (not over the budget page itself.

1

Option 1 (No plan): The user can choose a a local or long distance move, this prompts the user to input their TOTAL estimated budget, and it will auto fill the categories as a starting point



2

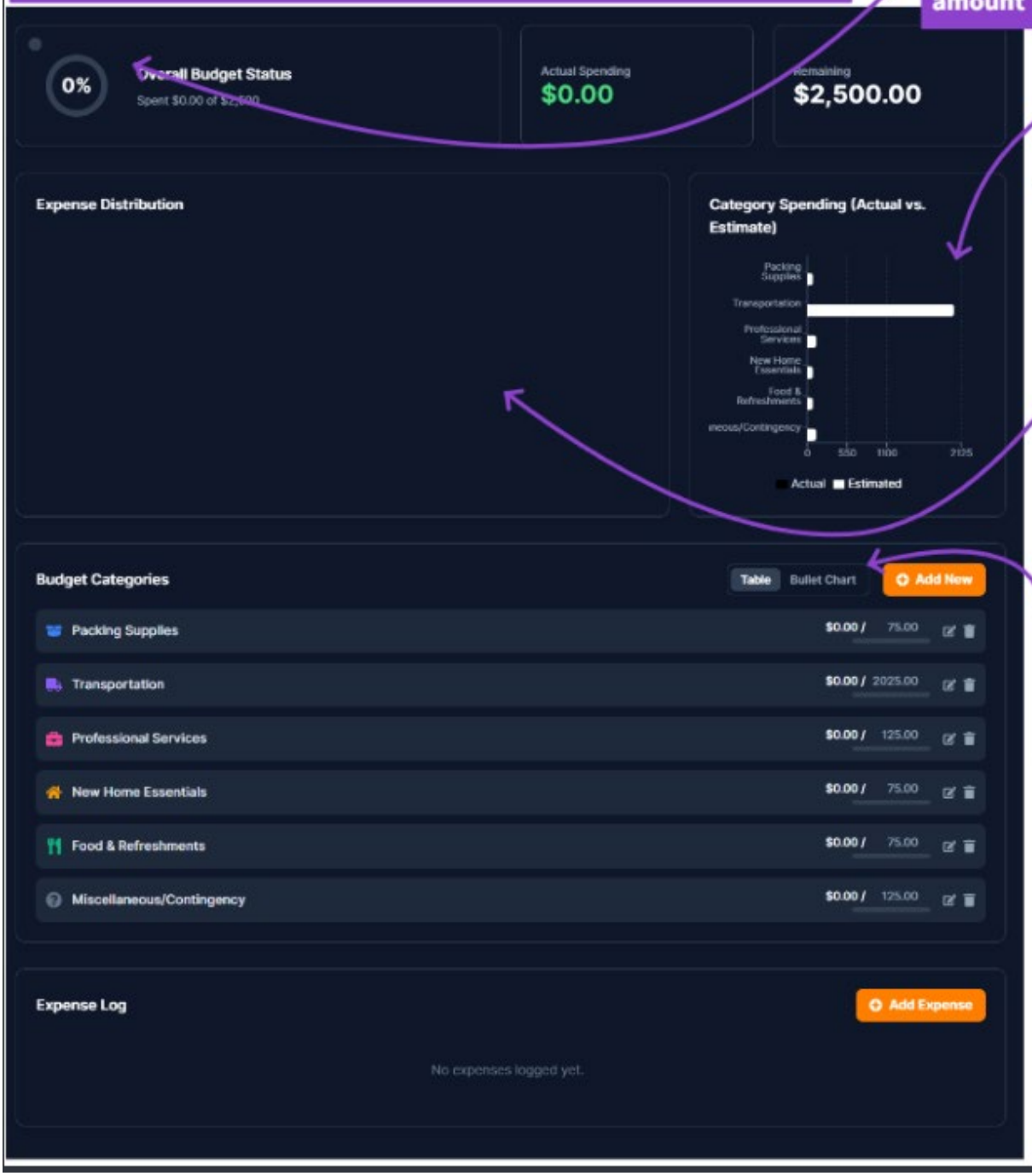
Regardless, the financial data the user provides is then transferred over to the actual budget page

Option 2 (preplanned): The user digitizes the budget they already created by adding the estimates into the categories (creating new ones as needed).

3

Until the user confirms their budget by tapping this button, they are not taken to the budgetpage

The data populates into the expenses, which will dynamically create and alter numerous data visualization tools. These tools include:



An overall circular graph that denotes when the overall estimate is close to and then exceeds the estimated amount

A bar graph that denotes when a category exceeds its estimate

And an expense tracker donut graph that displays the expenses and also denotes over budget based on category

The table and Bullet chart, directly integrated in the expenses section

Intended User Flow for Budget Setup

1. **Accessing the Budget Feature**
    - When the user navigates to the budget section (via a button or menu), the app checks if a budget has already been set.
    - **Logic Location:**
        - This check and the overall flow control are handled in

FinancialNavigator.tsx.

2. **Prompting for Setup (First Use)**
    - If no budget exists, a modal appears prompting the user to set up their budget.
    - The user is asked to:
        - Enter a total budget amount.
        - Select a move type (e.g., Local or Cross-State).
        - Optionally, enter category-specific budgets in advanced mode.
    - **Logic Location:**
        - The modal and form logic are in

BudgetSetup.tsx.

3. **Saving the Budget**
    - Upon submission, the selected budget and move type are passed to the parent component.
    - The app saves this information in its persistent state.
    - If this is the first setup, category budgets are pre-filled using templates that correspond to the selected move type.
    - **Logic Location:**
        - The saving and template application logic are in

FinancialNavigator.tsx (specifically, the

handleSetBudget function).

- State is managed and persisted using a custom hook in

hooks/usePersistentReducer.ts.

4. **Budget Usage in the App**
    - The saved budget and move type influence calculations, category budgets, and how expenses are tracked and displayed.
    - All calculations and displays reference the persisted state.
    - **Logic Location:**
        - Ongoing state and calculations are managed in

FinancialNavigator.tsx and related components.

- Budget templates and move type definitions are in

constants.tsx and

types.ts.

Design Intent

- The flow is designed to be user-friendly and guide users through necessary setup steps only when required (i.e., on first use).
- The use of move types allows the app to tailor budget templates and recommendations to the user's specific moving scenario.
- All logic is modularized: UI in

BudgetSetup.tsx, state and flow in

FinancialNavigator.tsx, and persistence in

hooks/usePersistentReducer.ts.